

状态的艺术

动态规划 · 8题 · C++ & Python 双语对照

小鲁进入了算法竞赛的核心——**动态规划(DP)**。DP不是一种算法，而是一种思想——将大问题拆成小问题，用小问题的解构造大问题的解。DP的精髓在于**状态定义**和**转移方程**。本章8题覆盖DP的四大类型：背包DP、线性DP、区间DP、记忆化搜索。学完本章，你才是一个真正的算法选手。

 **DP四大类型速查**

- **背包DP**：01(倒序)、完全(正序)。物品选或不选→二维压一维的关键是遍历顺序
- **线性DP**：LIS(一维)、LCS(二维)、编辑距离(二维)。字符串/序列问题的标准模型
- **区间DP**：石子合并——len从小到大，k分割区间。 $O(n^3)$ 三重循环模板
- **记忆化搜索**：DFS+备忘录=自顶向下DP。滑雪、树形DP。递归优雅但注意栈深度

NQ151 01背包问题 AcWing 2

小鲁终于进入了算法竞赛的巅峰领域——**动态规划(DP)**。"01背包：N件物品，每件有体积和价值。背包容量V——选哪些物品使总价值最大？"小华说，" $f[i][j]$ =前i件物品用j容量能获得的最大价值。每件选或不选： $f[i][j]=\max(f[i-1][j], f[i-1][j-v[i]]+w[i])$ 。这就是DP的核心——用子问题的解构造原问题的解。优化：将二维压成一维，j从大到小遍历。"

- 输入

第一行N,V。然后N行每行v_i w_i。
- 输出

最大总价值。
- 范围

0

输入	输出
4 5	8
1 2	
2 4	
3 4	
4 5	

思路 二维DP： $f[i][j]=\max(f[i-1][j], f[i-1][j-v]+w)$ 。一维优化： $f[j]=\max(f[j], f[j-v]+w)$ ，j从V倒序到v（防止重复使用）。时间 $O(NV)$ ，空间 $O(V)$ 。初始化 $f[0]=0$ （全0初值表示恰好装满？不——恰好装满需 $f[1..V]=-INF$ ）。

技巧 01背包的变形：完全背包(j正序)、多重背包(二进制拆分)。一维优化的关键是j倒序——保证每件物品只被选一次。DP的五步法：1)确定状态定义 2)初始化 3)转移方程 4)遍历顺序 5)答案位置。这个框架适用于所有DP问题。

C++

```
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 1007;
// 2维的解法
int n, m;
int f[N][N];
int v[N], w[N];

int main()
{
    cin >> n >> m; // 读入 N, V
    // 读入 v, w
    for (int i = 1; i <= n; i++) cin >> v[i] >> w[i];
    // 朴素的二维dp
    for (int i = 1; i <= n; i++)
        for (int j = 0; j <= m; j++)
        {
            f[i][j] = f[i-1][j];
            if (j >= v[i]) // 剩余体积大于第i项的体积
                f[i][j] = max(f[i][j], f[i-1][j-v[i]]+w[i]);
        }
    // 根据定义，从前n个物品中，选取总体积不超过m的选法集合 f[n][m] 就是答案
    cout << f[n][m] << endl;
}
```

Python

Python solution

NQ152 完全背包问题 AcWing 3

"01背包每件只能选一次——如果每件可以选无限次呢？"小华问。"那就是完全背包！"小鲁推导道，" $f[i][j] = \max(f[i-1][j], f[i][j-v] + w)$ ——注意第二项是 $f[i]$ 不是 $f[i-1]$ ，因为选了之后 i 仍可选。优化到一维： j 正序从 v 到 V ——正好体现'可重复选'的特点。01倒序、完全正序——两行代码的区别背后是完全不同的DP语义。"

输入 第一行 N, V 。然后 N 行每行 $v_i w_i$ 。

输出 最大总价值。

范围 0

输入	输出
4 5	10
1 2	
2 4	
3 4	
4 5	

思路 完全背包一维：for i: for j from v to V: $f[j] = \max(f[j], f[j-v] + w)$ 。 j 正序遍历！因为同一件物品可以被多次选取——正序遍历时 $f[j-v]$ 是已更新过的第 i 层状态，允许重复选。时间 $O(NV)$ ，空间 $O(V)$ 。

技巧 完全背包正序、01背包倒序——背后是同一个转移方程在不同遍历顺序下的不同效果。理解这一点，多重背包的二进制拆分也就懂了。完全背包的初始化同样要注意是否恰好装满。

C++

```
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 1007;

int n,m;
int f[N][N];
int v[N],w[N];

int main()
{
    cin >>n >>m;
    for (int i = 1; i <= n; i++) cin>>v[i]>>w[i];

    //完全背包
    // 1.f[i][j] = f[i-1][j];(不选物品i)
    // 2.f[i][j]= max(f[i-1][j], f[i][j-v[i]]+w[i])(选若干个物品i)
    for (int i = 1; i <= n; i++)
        for (int j=0; j <= m; j++)
        {
            f[i][j]=f[i-1][j];
            if (j >= v[i])
                f[i][j] = max(f[i-1][j], f[i][j-v[i]]+w[i]);
        }

    cout<<f[n][m]<<endl;
    return 0;
}
```

Python

Python solution

NQ153 数字三角形 AcWing 898

"一个三角形数塔——从顶部出发，每次可以向下走左或右，求到达底层的最大路径和。"小华画出三角，"这是线性DP的入门—— $f[i][j]$ =三角形第*i*行第*j*列到达底层的最大和。自底向上推： $f[i][j]=\max(f[i+1][j], f[i+1][j+1])+a[i][j]$ 。顶部 $f[0][0]$ 就是答案。也可以自顶向下推——但自底向上不需要处理边界。"

输入

第一行n。然后n行三角形数字。

输出

最大路径和。

范围

$1 \leq n \leq 500, -10000 \leq \text{值} \leq 10000$

输入

5
7
3 8
8 1 0

输出

30

```
2 7 4 4
4 5 2 6 5
```

思路 自底向上DP：从倒数第二行开始， $f[i][j] = \max(f[i+1][j], f[i+1][j+1]) + a[i][j]$ 。顶部 $f[0][0]$ 为答案。时间 $O(n^2)$ ，空间 $O(n^2)$ (滚动数组可 $O(n)$)。这是最短路径DP和三角形DP的最基本形态。

技巧 自底向上避免了边界检查——每一层总有下一层。自顶向下需要处理 $j=0$ 和 $j=i$ 的边界。这是一个重要经验：DP的迭代方向选择直接影响代码复杂度。数字三角形也是树形DP和网格DP的简化版。

C++

```
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 507;

int n;
int w[N][N], f[N][N]; //三角矩阵依旧用2维存储

int main()
{
    //读入n, w
    cin >> n;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= i; j++)
            cin >> w[i][j];

    //初始化最底层的f[n][j]
    for (int i = 1; i <= n; i++) f[n][i] = w[n][i];

    //用dp从底层往上计算每个f[i][j]的值
    for (int i = n-1; i; i--)
        for (int j = 1; j < n; j++)
            f[i][j] = max(f[i+1][j] + w[i][j], f[i+1][j+1] + w[i][j]);

    cout << f[1][1] << endl;
    return 0;
}
```

Python

```
# Python solution
```

NQ154 最长上升子序列 AcWing 895

"LIS——最长上升子序列。给一个数组，找最长的严格递增的子序列（可以不连续）。"小华说，"朴素DP： $f[i] = \text{以} a[i] \text{结尾的 LIS 长度} = \max(f[j] + 1) \text{ for } j$

输入 第一行 N 。第二行 N 个整数。

输出 LIS长度。

范围 $1 \leq N \leq 100000$

输入
7
3 1 2 1 8 5 6

输出
4

思路 朴素 $O(n^2)$: 对每个 i 遍历 j

技巧 LIS的优化是用贪心维护'最小结尾值'——保证tails数组始终是递增的, 可以用二分。这个优化思路也用于LIS的变种 (LNDS/带权LIS)。DP+贪心+二分=算法设计中最经典的技术融合。

C++

```
#include <iostream>
#include <algorithm>
#include <cstring>
using namespace std;
// f[i] 定义为以s[i] 结尾的最长的子序列
// 集合: 所有以第i个数结尾的上升子序列集合
// 属性: Max 上升子序列长度的最大值
const int N = 10007;
int n;
int a[N], f[N];
// 状态计算
int main()
{
    cin >> n;
    for (int i = 1; i <= n; i++)
        cin >> a[i]; // 从1开始初始化

    for (int i = 1; i <= n; i++)
    {
        f[i] = 1; // a[i] 只有一个元素, 所以f[i]=1;
        for (int j = 1; j < i; j++) // 求i之前的f[j]
            if (a[j] < a[i])
                f[i] = max(f[i], f[j] + 1);
    }

    int res = 1;
    for (int i = 1; i <= n; i++)
        res = max(res, f[i]);
    cout << res << endl;
    return 0;
}
```

Python

Python solution

NQ155 最长公共子序列 AcWing 897

"LCS——两个字符串的最长公共子序列。"小华说, "DP经典中的经典—— $f[i][j]$ =A前 i 个和B前 j 个的LCS长度。若 $A[i]=B[j]$ 则 $f[i][j]=f[i-1][j-1]+1$; 否则 $f[i][j]=\max(f[i-1][j], f[i][j-1])$ 。二维DP的范式。LCS的应用贯穿生物信息学 (DNA比对)、diff工具、拼写纠错——是字符串DP的核心模型。"

输入 第一行 n, m 。第二行A串, 第三行B串。

输出 LCS长度。

范围

$1 \leq n, m \leq 1000$

输入

4 5
acbd
abedc

输出

3

思路 二维DP: $f[i][j]$ 定义如上。双重循环填充: 若匹配则左上+1, 否则取上/左的最大值。最终答案在 $f[n][m]$ 。时间 $O(nm)$, 空间 $O(nm)$ 可压到 $O(\min(n,m))$ 滚动数组。这是编辑距离的前身——只是代价函数不同。

技巧 LCS的转移图示: 方向箭头——左上(匹配)、上(跳过A[i])、左(跳过B[j])。空间优化: 只用两行交替($f[cur]$ 和 $f[prev]$)。LCS可以扩展为带权版本、多串版本——理解基本模型后扩展到海量变种。

C++

```
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
    {
        f[i][j] = max(f[i-1][j], f[i][j-1]);
        if (a[i]==b[j]) f[i][j] = max(f[i][j], f[i-1][j-1]+1);
    }

cout<<f[n][m]<<endl;
return 0;
```

Python

Python solution

NQ156 石子合并 AcWing 282

"区间DP的经典——N堆石子排成一排, 每次合并相邻两堆, 代价为两堆石子数之和, 求合并成一堆的最小总代价。"区间DP范式: $f[l][r]$ =合并 $[l,r]$ 区间的最小代价= $f[l][k]+f[k+1][r]+sum[l..r]$ 。三重循环——len从小到大($1 \rightarrow n$)、l从0开始、k在 $[l,r]$ 分割。最终答案 $f[0][n-1]$ 。区间DP是理解DP最优子结构的最佳入口。"

输入

第一行N。第二行N个整数表示每堆石子数。

输出

最小总代价。

范围

$1 \leq N \leq 300$

输入

4
1 3 5 2

输出

22

思路 区间DP模板: for len=2 to n: for l=0 to n-len: r=l+len-1; $f[l][r]=INF$; for k=l to r-1: $f[l][r]=\min(f[l][r], f[l][k]+f[k+1][r]+s[r+1]-s[l])$ 。前缀和 $s[i]$ =前i堆的总和快速求区间和。 $O(n^3)$, $n \leq 300$ 时OK。

技巧 区间DP三要素: len从小到大(保证子区间已算好)、l遍历起点、k遍历分割点。前缀和 $O(1)$ 求区间和。这题可以用平行四边形优化到 $O(n^2)$ 但不在本课范围。理解这个三层循环模式, 所有区间DP问题都是它的变

体。

C++

```
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 307;

int n;
int s[N]; // 前缀和
int f[N][N]; // 所有将[i,j]合并成一堆的方案数

int main()
{
    cin>>n;
    for (int i = 1; i <= n; i++) cin>>s[i], s[i] += s[i-1]; // 计算前缀和

    // 区间DP, 先枚举长度, 后枚举端点
    for (int len = 2; len <= n; len++)
        for (int i = 1; i+len-1 <= n; i++) // 区间左端点
        {
            // 区间右端点
            int j = i + len - 1;
            f[i][j] = 1e9; // 初始化为无穷大
            for (int k = i; k < j; k++) // 区间dp, 枚举分隔点
                f[i][j] = min(f[i][j], f[i][k] + f[k+1][j] + s[j] - s[i-1]);
        }

    cout<<f[1][n]<<endl;
    return 0;
}
```

Python

Python solution

NQ157 最短编辑距离 AcWing 902

"把字符串A变成字符串B——每次可以增、删、改一个字符，最少操作次数？"这是编辑距离(Levenshtein Distance)。" $f[i][j]$ =A前i个变B前j个的最小操作数。增： $f[i][j-1]+1$ ；删： $f[i-1][j]+1$ ；改： $f[i-1][j-1]+1$ ($A[i] \neq B[j]$)。三重min。编辑距离是所有拼写检查、DNA比对、文档diff的底层算法——LCS的泛化。"

输入

第一行n和A串。第二行m和B串。

输出

最小操作次数。

范围

$1 \leq n, m \leq 1000$

输入

4
AGTCTGACGC
3
AGTAAGTAGGC

输出

(见OJ实际输出)

思路 编辑距离DP: $f[i][j]$ 如上。初始化 $f[i][0]=i$ (全删), $f[0][j]=j$ (全增)。三重min转移。时间 $O(nm)$, 空间 $O(nm) \rightarrow$ 滚动数组 $O(\min(n,m))$ 。与LCS的区别在于可以有替换操作且替换 $cost=1$ 而非跳过代价。

技巧 编辑距离的初始化不要忘——空串变 j 字符= j 次增加。三个操作对应三个方向: 上=删、左=增、左上=改。每个方向+1(替换是否+1取决于字符是否相同)。这个DP是所有字符串距离算法的根基。

C++

```
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;
const int N = 1007;

int n,m;
char a[N],b[N];
int f[N][N]; // 所有將a[i]變為b[i]的操作步驟

int main()
{
    cin>>n>>a+1; // 下標從1開始
    cin>>m>>b+1;

    // 初始化
    for (int i = 0; i < N; i++) f[i][0] = i; // 把a[i]變為空串
    for (int i = 0; i < N; i++) f[0][i] = i; // 把空串變為b[i]

    // DP
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
        {
            f[i][j] = min(f[i-1][j] + 1, f[i][j-1] + 1);
            if (a[i]==b[j]) f[i][j] = min(f[i][j], f[i-1][j-1]);
            else f[i][j] = min(f[i][j], f[i-1][j-1] + 1);
        }

    cout<<f[n][m]<<endl;

    return 0;
}
```

Python

```
# Python solution
```

NQ158 滑雪 AcWing 901

"记忆化搜索——DP和搜索的完美结合。"华说, "给定一个二维矩阵表示雪山高度, 从某点开始滑只能从高往低滑到四个方向。求最长滑行路径。 $dp[i][j]$ =从 (i,j) 出发的最长路径= $1+\max(dp[nx][ny])$ 对所有合法邻居(高度更低)。用DFS+备忘录——递归第一次算, 后续直接查表。记忆化=Top-down DP, 递推=Bottom-up DP。"

输入

第一行R,C。然后 $R \times C$ 矩阵。

输出

最长滑行长度。

范围

$1 \leq R, C \leq 300$

输入

```
5 5
1 2 3 4 5
16 17 18 19 6
15 24 25 20 7
14 23 22 21 8
13 12 11 10 9
```

输出

25

思路 记忆化DFS: $dp[i][j] = 1 + \max(\text{dfs}(nx, ny) \text{ for 所有高度更低的邻居})$ 。每次dfs先检查 $dp[i][j]$ 是否已算(不等于-1), 已算直接返回。每个状态算一次, $O(RC)$ 。比纯DFS快是因为避免了重复搜索——这正是DP的威力。

技巧 记忆化搜索=自顶向下DP。优势: 按需计算(只计算能到达的状态)。劣势: 递归栈深度限制。DFS中四个方向的顺序不影响结果。Python递归可能需要`sys.setrecursionlimit(1000000)`。这道题展示了搜索→DP的自然过渡。

C++

```

#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 307;

int g[N][N]; // 地图高度
int f[N][N]; // 从[i,j]开始滑的最长的轨迹

int n, m; // 宽高

int dx[4] = {-1, 0, 1, 0}, dy[4] = {0, 1, 0, -1};

int dp(int x, int y)
{
    // 记忆化搜索
    if (f[x][y] != -1) return f[x][y];
    // 否则, 从x,y开始滑
    f[x][y] = 1; // 当前位置至少是1格
    // 分上下左右四个方向滑雪
    for (int i = 0; i < 4; i++)
    {
        int a = x + dx[i], b = y + dy[i];
        if (a < 1 || a > n || b < 1 || b > m) continue; // 越界
        if (g[a][b] >= g[x][y]) continue; // 不能走
        // 四个方向深搜, 更新f[a][b], 取最大值
        f[x][y] = max(f[x][y], dp(a, b) + 1);
    }

    return f[x][y]; // 返回f[x][y]
}

int main()
{
    cin >> n >> m;
    // 读入高度图
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            cin >> g[i][j];

    memset(f, -1, sizeof f); // 初始化为-1, 表示该点未被访问

    // 把每个点作为起点深搜
    int res = 0;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            res = max(res, dp(i, j));

    cout << res << endl;
    return 0;
}

```

Python

Python solution

本章知识点总结

- **DP五步法**: 定义状态→初始化→转移方程→遍历顺序→答案位置。模板先于变种
- **背包进化**: 01倒序、完全正序——遍历顺序决定是否重复选

- **字符串DP三连**：LIS(一维+二分优化)、LCS(二维+状态转移)、编辑距离(三维min)
- **区间DP**：len从小到大、l枚举起点、k枚举分割。前缀和加速区间和查询
- **记忆化搜索**：DFS+备忘录=优雅的Top-down。滑雪、树形DP的标准写法

— 第15章完 · 共8题 · 自强不息 —